Portfolio Assessment 3

Machine Learning and Deep Learning

Electrical grid stability classification

Student: Hoang Huu Hoan
Student ID: 105732188
Unit: COS40007 Artificial Intelligence Engineering
Task number: Portfolio Assessment 3
Submission date: 27 October 2026

GitHub repository

Acknowledgement of Country

This report recognises the Wurundjeri People of the Kulin Nation as the Traditional Owners of the Country where Swinburne's Melbourne campuses stand. Respect is offered to Elders past and present and to the enduring cultures and custodianship of Aboriginal and Torres Strait Islander peoples across Australia.

Part A Traditional machine learning

The cleaned electrical-grid dataset has 10,000 rows and 14 columns. The target stabf is categorical: stable is encoded 0 and unstable 1, so accuracy and F1 are appropriate. Regression R squared, RMSE and MAE are not applicable. There are 3,620 stable and 6,380 unstable cases; always predicting unstable would achieve 63.8% accuracy.

Eleven features are retained: tau1 to tau4, p2 to p4, and g1 to g4. The continuous stab value is excluded because its sign determines the target. The redundant p1 column is also excluded because it equals the negative sum of p2, p3 and p4. No missing values or duplicate rows were found, and all rows were retained.

A stratified 70% development and 30% test split uses seed 42. Shapes are X development (7000, 11), y development (7000,), X test (3000, 11) and y test (3000,). StandardScaler is fitted only on the training portion at each stage. Cross-validation uses pipelines so each fold learns its own scaler.

Model	Main settings	CV accuracy mean and SD	CV weighted F1 mean and SD	Test accuracy	Test weighted F1
RF	300 trees; seed 42	91.69% ± 0.65 pp	0.9163 ± 0.0064	92.73%	0.9268
MLP	64/32/16 ReLU; Adam 0.001; max 500 iterations	95.41% ± 0.59 pp	0.9541 ± 0.0059	96.47%	0.9646

Five shuffled stratified folds use seed 42 on the 7,000 development rows. Weighted F1 means and sample standard deviations are 0.9163 ± 0.0064 for Random Forest and 0.9541 ± 0.0059 for MLP. Fold SD describes fold variation, not a confidence interval. The final MLP stops after 128 iterations without convergence warnings. MLP is the strongest traditional model by both development CV and test accuracy.

Dataset check	Observed result
Rows and original columns	10,000 rows and 14 columns
Retained predictors	11 ordered numeric features; all 10,000 rows retained
Missing and duplicate checks	0 missing cells; 0 duplicate rows; 0 duplicate predictor rows
Numeric and target checks	All numeric values finite; target agrees with stability sign; power balance confirmed

Part B Data preparation and architecture

This dataset contains independent tabular rows, with no sequence or image structure. No sequence creation, reshaping to time steps, or image preprocessing is required. Both core ANNs consume the same 11 ordered numeric features and predict the same two classes.

Partition	X shape	y shape	Stable	Unstable
development	(7000, 11)	(7000,)	2534	4466
ANN fit	(5600, 11)	(5600,)	2027	3573
ANN validation	(1400, 11)	(1400,)	507	893
test	(3000, 11)	(3000,)	1086	1914

The validation reserve is 20% of development data, selected by a stratified split with seed 43. Its scaler is fitted on the 5,600 fit rows only. Validation and test rows never fit that scaler. After the recipe is frozen, a separate common scaler is fitted to all 7,000 development rows and reused by all four final models. Features are standardised; class labels remain integers.

Layer	Output shape	Activation or rate	Parameters
Input	(None, 11)	Numeric features	0
Dense	(None, 64)	ReLU	768
Dropout	(None, 64)	0.2	0
Dense	(None, 32)	ReLU	2080
Dropout	(None, 32)	0.2	0
Dense	(None, 16)	ReLU	528
Output Dense	(None, 2)	Softmax	34
Trainable parameters			3410
Non trainable parameters			0
Total parameters			3410

Model summary reproduced from notebook cell 10. None denotes a variable batch dimension. All 3,410 network parameters are trainable; dropout adds no parameters.

ReLU supports nonlinear feature interactions. The shrinking hidden layers compress the representation. Dropout randomly suppresses 20% of hidden activations during training, which can reduce overfitting but cannot guarantee its prevention. Softmax gives two class probabilities; argmax produces the class label. A tie selects class 0. Adam begins at 0.001; sparse categorical crossentropy matches integer labels, and accuracy is monitored. Layer width and dropout were fixed, so these experiments do not isolate their individual causal effect.

Part B Training and configuration selection

A batch-32 pilot first runs for 10 epochs. Its lowest validation loss is 0.1830 at epoch 8, with validation accuracy 91.50%. Fresh batch-size trials then run for at most 50 epochs, using the same fit and validation rows, the same fit-only scaler and seed 42. Batch size is the deliberate trial difference; training starts from a fresh model each time.

Phase and batch	Epochs run	Best epoch	Best validation loss	Accuracy at best loss	Minimum LR
Pilot 32	10	8	0.1830	91.50%	0.0010000
Trial 16	50	40	0.1055	96.07%	0.0000625
Trial 32	49	39	0.1070	95.86%	0.0001250
Trial 64	50	46	0.1080	95.93%	0.0005000

EarlyStopping monitors validation loss with patience 10, min_delta 0 and restore_best_weights enabled. ReduceLROnPlateau monitors the same loss, halves the learning rate after five plateau epochs, uses min_delta 0, and has a minimum rate of 0.000001. Restored-model evaluation is checked against the lowest validation loss recorded in history.

Batch 32 reaches its best epoch at 39 and stops at 49 after ten epochs without improvement. Batches 16 and 64 reach the 50-epoch cap; a cap does not mean their best weights came from epoch 50. Learning rates are reduced four, three and one times for batches 16, 32 and 64 respectively.

ANN-1 is the batch-32 recipe, and ANN-2 is batch 64. Batch 16 is an additional development experiment and has the lowest validation loss. It is retained in the trial table; no final test result is claimed for it. These required two core configurations were fixed before final test inference. A batch-16 final model is a possible future, predeclared experiment, not a change justified by this test comparison.

For the final comparison, every model is refitted on all 7,000 common-scaled development rows. The ANNs use the selected 39 and 46 epochs with constant Adam 0.001 and no validation callbacks. These are newly fitted final models. The reduced-learning-rate development models and the final fixed-rate models are different fitted objects; their histories must not be mixed.

```
def train_trial(batch, max_epochs):
    model = build_ann()
    reduce_lr = keras.callbacks.ReduceLROnPlateau(monitor='val_loss',factor=.5,patience=5,
                                                  min_delta=0.0,min_lr=1e-6,verbose=0)
    early = keras.callbacks.EarlyStopping(monitor='val_loss',patience=10,
                                          min_delta=0.0,restore_best_weights=True)
```

S04. Training callback settings

Part B Learning curves

The plotted development curves describe the fit and validation experiment, not the final 7,000-row refit. Lower cross entropy indicates better probability assignments; validation loss is used to choose epochs. Neither the test set nor the final accuracy bar chart is used to choose an epoch.

For batch 32, fit loss falls from 0.5096 at the first epoch to 0.1077 at the last recorded epoch; the best validation loss is 0.1070. For batch 64, fit loss falls from 0.5522 to 0.1236, and the best validation loss is 0.1080. The overall loss trends support learning, with ordinary epoch-to-epoch fluctuations rather than a requirement for every point to decrease.

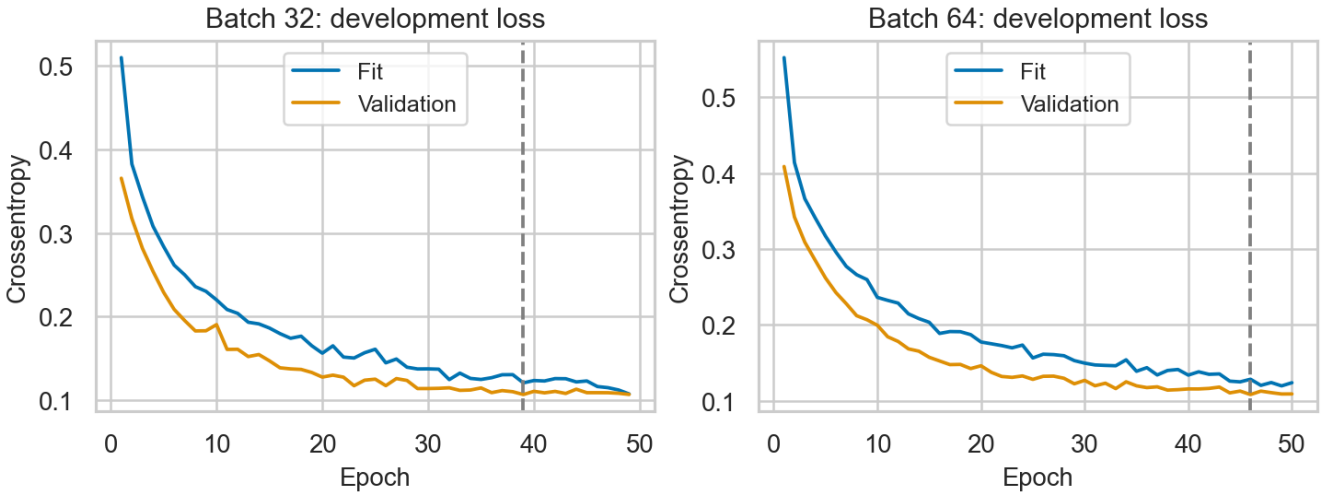


Figure 1. ANN development loss. Dashed markers show the selected epochs 39 and 46. These are trial histories, not the final 7,000-row refit histories.

Training and validation accuracy

Fit accuracy rises from 74.89% to 95.77% for batch 32 and from 72.00% to 94.98% for batch 64. Validation accuracy at the minimum-loss epoch is 95.86% and 95.93%, respectively. Validation accuracy can exceed fit accuracy because dropout is disabled during validation and the samples differ. This pattern alone is not proof of leakage.

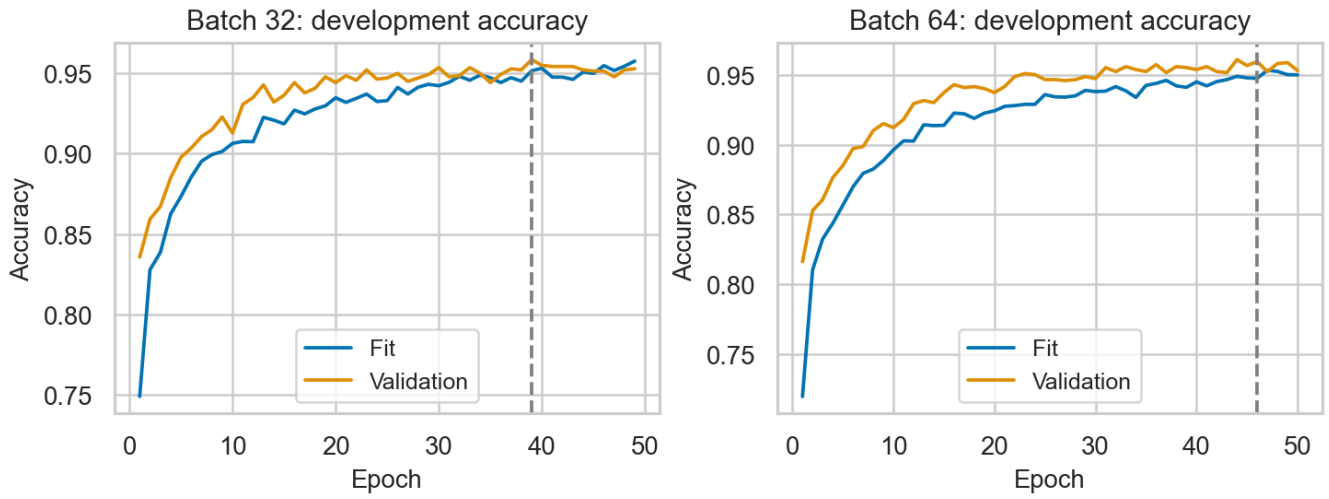


Figure 2. ANN development accuracy for batches 32 and 64 using the same fit and validation partitions.

Learning rate schedule

The learning-rate curves make the plateau response visible. Smaller batches make more updates per epoch: approximately 350 for batch 16, 175 for batch 32 and 88 for batch 64 on 5,600 rows. Equal epoch caps therefore do not provide equal update budgets. Different stopping epochs and learning-rate schedules also limit causal conclusions about batch size alone.

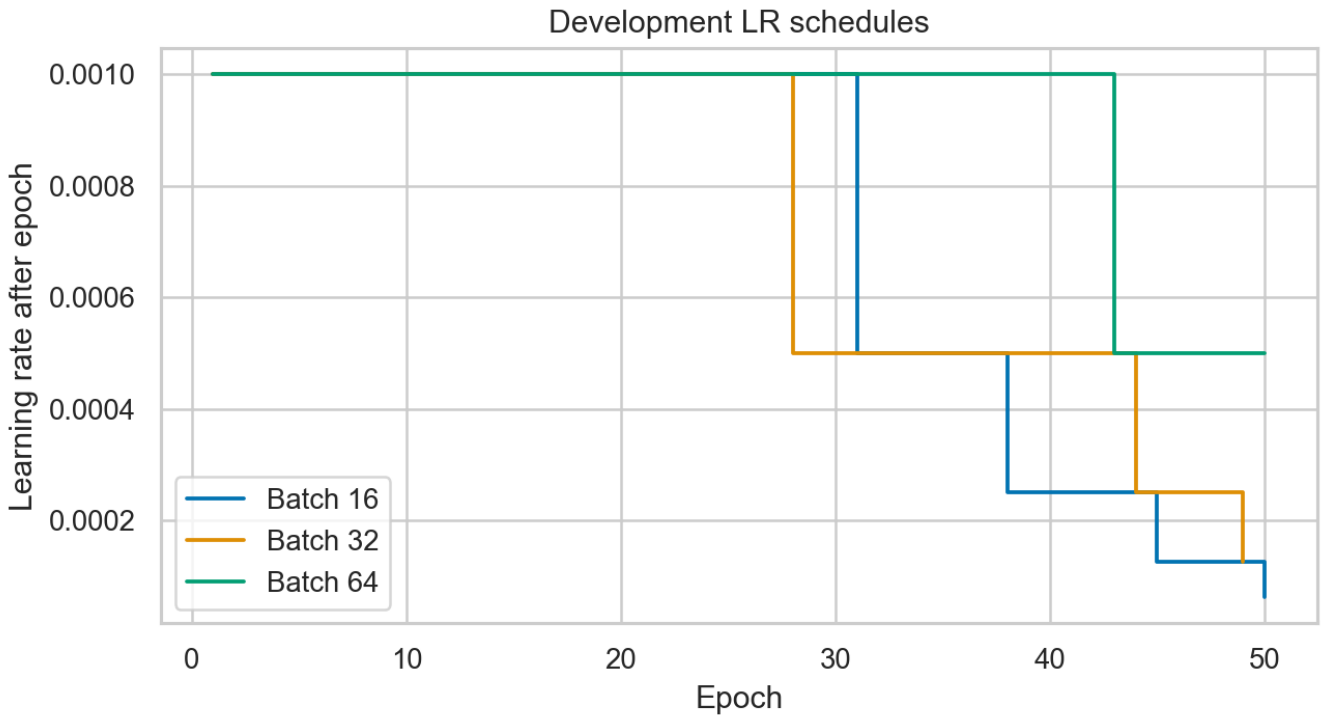


Figure 3. Development learning rates for batches 16, 32 and 64. Plateau callbacks halve the learning rate.

Part B Classification evaluation

All results below use the same 3,000 fixed test rows and final saved model recipes. Weighted F1 accounts for class support; the per-class report reveals performance that a single accuracy value can hide. Test crossentropy is evaluated for each Keras ANN. ANN five-fold cross-validation was not performed and is not implied by the holdout validation results.

Model	Test accuracy	Weighted F1	Test loss	Unstable recall
ANN-1	95.57%	0.9552	0.1023	98.80%
ANN-2	96.57%	0.9655	0.0915	98.59%

Model and class	Precision	Recall	F1	Support
ANN-1 stable	0.9770	0.8987	0.9362	1086
ANN-1 unstable	0.9450	0.9880	0.9660	1914
ANN-1 macro avg	0.9610	0.9433	0.9511	3000
ANN-1 weighted avg	0.9566	0.9557	0.9552	3000
ANN-2 stable	0.9740	0.9300	0.9515	1086
ANN-2 unstable	0.9613	0.9859	0.9734	1914
ANN-2 macro avg	0.9676	0.9580	0.9625	3000
ANN-2 weighted avg	0.9659	0.9657	0.9655	3000

Confusion matrix rows are true stable and unstable; columns are predicted stable and unstable. ANN-1 has counts [[976, 110], [23, 1891]], and ANN-2 has [[1010, 76], [27, 1887]]. ANN-1 misses fewer unstable cases, but classifies more stable cases as unstable. ANN-2 improves stable recall, which raises overall accuracy. The class supports are 1,086 stable and 1,914 unstable.

Confusion matrices

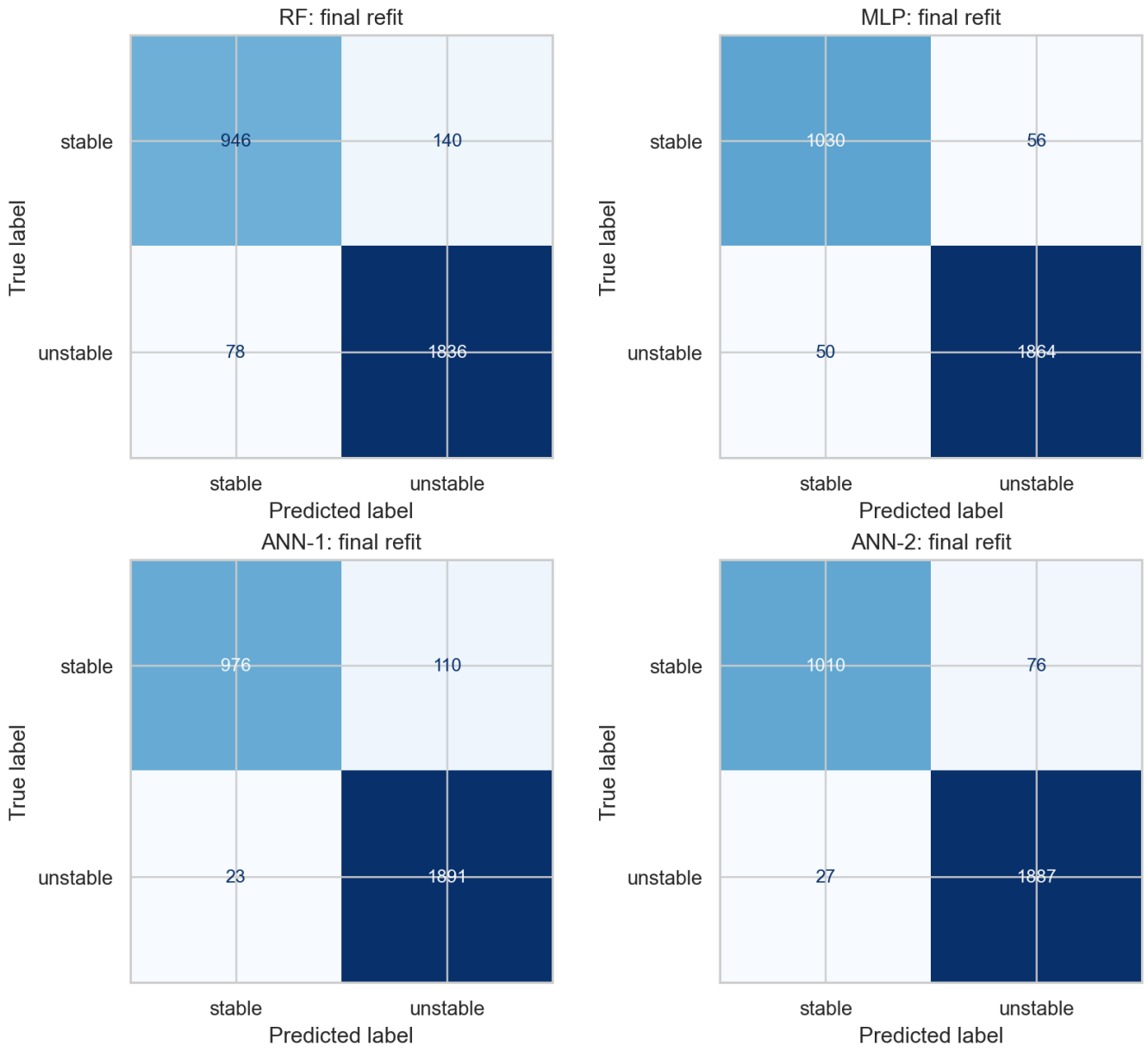


Figure 4. Final confusion matrices on the same 3,000 test cases. Rows are true labels and columns are predicted labels.

The best traditional MLP achieves 96.47% accuracy and weighted F1 0.9646. ANN-2 achieves 96.57% and 0.9655, while ANN-1 achieves 95.57% and 0.9552. Deep learning therefore improves the primary metric only for ANN-2, by a small observed margin; it is not uniformly superior.

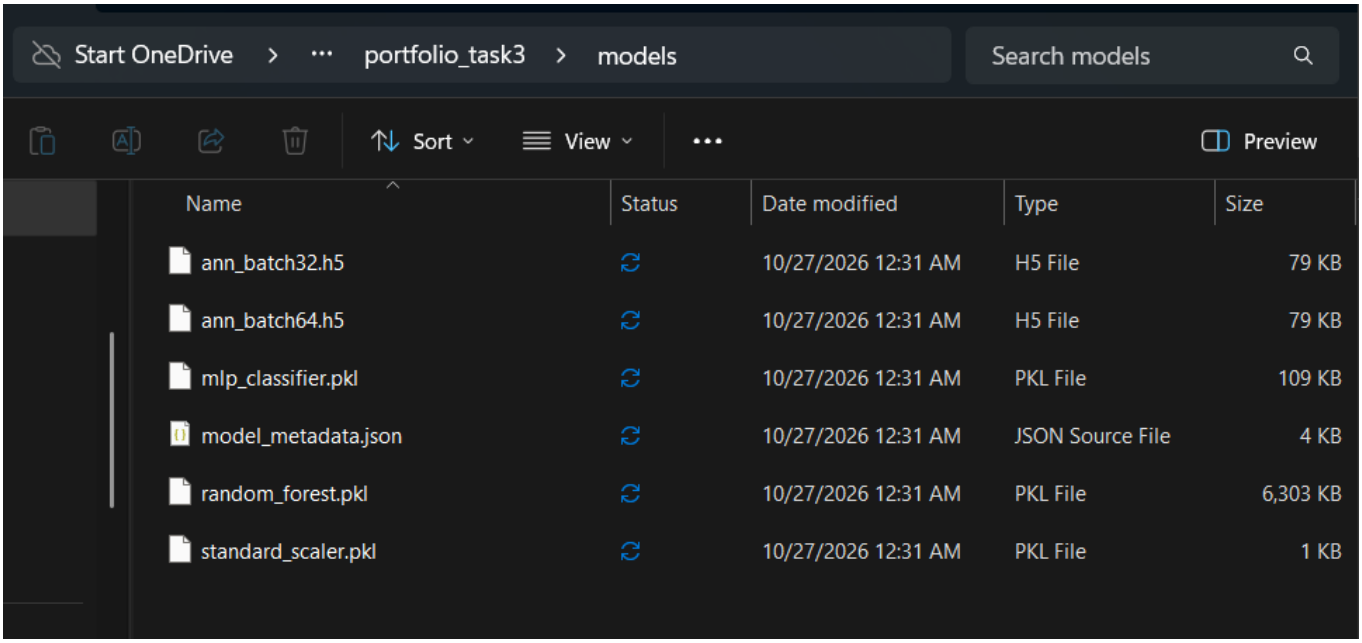
Part B Saved models and verification

Artifact relative to portfolio_task3	Purpose
models/ann_batch32.h5	Whole final ANN-1 model; 39-epoch refit.
models/ann_batch64.h5	Whole final ANN-2 model; 46-epoch refit.
models/standard_scaler.pkl	Common scaler fitted to the 7,000 development rows.
models/random_forest.pkl and mlp_classifier.pkl	Compressed joblib pipelines containing fitted scaling and traditional estimators; accept raw ordered features.
results/ann_batch32_development_history.pkl and ann_batch64_development_history.pkl	Validation trial histories used for the core learning curves.
results/ann_batch32_final_history.pkl and ann_batch64_final_history.pkl	Separate histories for the final refits.
results/ann_pilot_history.pkl and ann_batch16_development_history.pkl	Pilot and extra batch-16 development history.
models/model_metadata.json	Feature order, class map, data hash, versions, architecture and final recipe.

The H5 files contain model configuration, training configuration, weights and optimiser state. Both saved models have the expected two-class architecture. The scaler and traditional pipelines use joblib-compressed pickle format and should be loaded with joblib.load. Raw data must follow the documented order of the 11 predictors. Scaling is applied once: directly for the ANNs, internally by the traditional pipelines.

The independent verify_artifacts.py check reconstructs the test split and evaluates all 3,000 raw test rows for every model. It checks saved labels, probabilities, metrics, fitted scaler values and configuration metadata. This check passed again during this report audit. Reloaded class predictions matched exactly; probabilities matched within absolute and relative tolerances of 0.000001.

Earlier verification also executed all ten notebook code cells in a fresh environment without error outputs and reproduced the numerical results. A separate GitHub clone passed the dataset and artifact checks. These are execution and reproducibility checks, not evidence of real-grid safety or unseen-population generalisation.



	path	bytes	sha256
0	models/ann_batch32.h5	80376	9b3a78f2447ff56ae27b6338f0777ea26db8d048a13f1d...
1	models/ann_batch64.h5	80376	55b481ffaecc07dfcfcc8695eb4f3507c6872bad80b457...
2	models/mlp_classifier.pkl	110758	7aa077fc8e2fb698d9c47d38ad6b394187a19e2baa53ea...
3	models/model_metadata.json	3553	ae4f3f46c792dbd961733d420436d6233733924dbce2fc...
4	models/random_forest.pkl	6453418	643579152bdf8481b71dff8e3f38be98c55de71b104b09...
5	models/standard_scaler.pkl	842	99a07d918f6a6fbc5b5e74f091870dfcecfb830639b527...
6	results/ann_batch16_development_history.pkl	2817	92211de122f8be7d55545295bfe1189e1bf5bf06e87c30...
7	results/ann_batch32_development_history.pkl	2763	28a7d63cf14b8495c913dd49e53ce3375e1a8f6eac3c82...
8	results/ann_batch32_final_history.pkl	744	440c3ccf8ccdb16010b58e42b4c120fd604307cfbce3d6...
9	results/ann_batch64_development_history.pkl	2817	8c4b3d80253d8c128a905c075bd0a32a6c9dd771540d60...
10	results/ann_batch64_final_history.pkl	870	2f768f6341dd35c9690211c8323dbeda25d86fafdf4a18...
11	results/ann_pilot_history.pkl	657	62f38afa55d18ff61400922219dec8faf71be20a9d60cc...

S11. Saved files and independent reload check

Evidence files: results/artifact_verification.json, fresh_environment_verification.json, fresh_reproduction_verification.json, model_artifact_manifest.json and remote_verification.json. No new training or model selection was performed for this explanation document.

Part C Master comparison and reflection

Type and model	Accuracy	Weighted F1	Balanced accuracy	Unstable as stable
Traditional ML RF	92.73%	0.9268	91.52%	78
Traditional ML MLP	96.47%	0.9646	96.12%	50
Deep Learning ANN-1	95.57%	0.9552	94.33%	23
Deep Learning ANN-2	96.57%	0.9655	95.80%	27

ANN-2 is the best observed model by the specified primary metric, accuracy. It classifies 2,897 of 3,000 cases correctly versus 2,894 for MLP: a difference of three cases, or 0.10 percentage points. MLP has the highest balanced accuracy, 96.12%, while ANN-1 makes the fewest unstable-as-stable errors, 23. The preferred model changes with the operational objective.

How architecture choices affected performance

The 64/32/16 dense network can represent nonlinear interactions among response times, powers and gains. Dropout encourages distributed representations. Two Softmax outputs match the categorical target. These choices are structurally suitable, but no width or dropout ablation was run, so the data do not show how much each choice changed performance. Batch-size trials changed optimisation behaviour; they did not change the layer architecture.

Whether deep learning outperformed traditional models

ANN-2 narrowly exceeds the strongest traditional MLP in accuracy and weighted F1; ANN-1 does not. Different optimisation and regularisation can explain plausible differences, but the comparison does not establish a causal explanation or statistical superiority. The scikit-learn MLP is itself a neural network, so the coursework groups distinguish implementations and training recipes rather than entirely unrelated model families.

Further improvements with more time

Use a genuinely new external or simulation-regime holdout, repeat development across seeds, and tune within development data only. Predeclare comparisons of dropout, widths and learning-rate schedules; include batch 16 in a future final comparison if justified independently. Assess probability calibration and cost-based thresholds using development data, then evaluate once on new data. Report uncertainty and training cost alongside accuracy.

Comparison visualisations and deployment

Figure 5 compares all four core configurations and includes the 63.8% majority baseline. The MLP and ANN-2 bars are almost identical at this scale. The three-case difference is stated in the caption so the small observed gap is not presented as a decisive win.

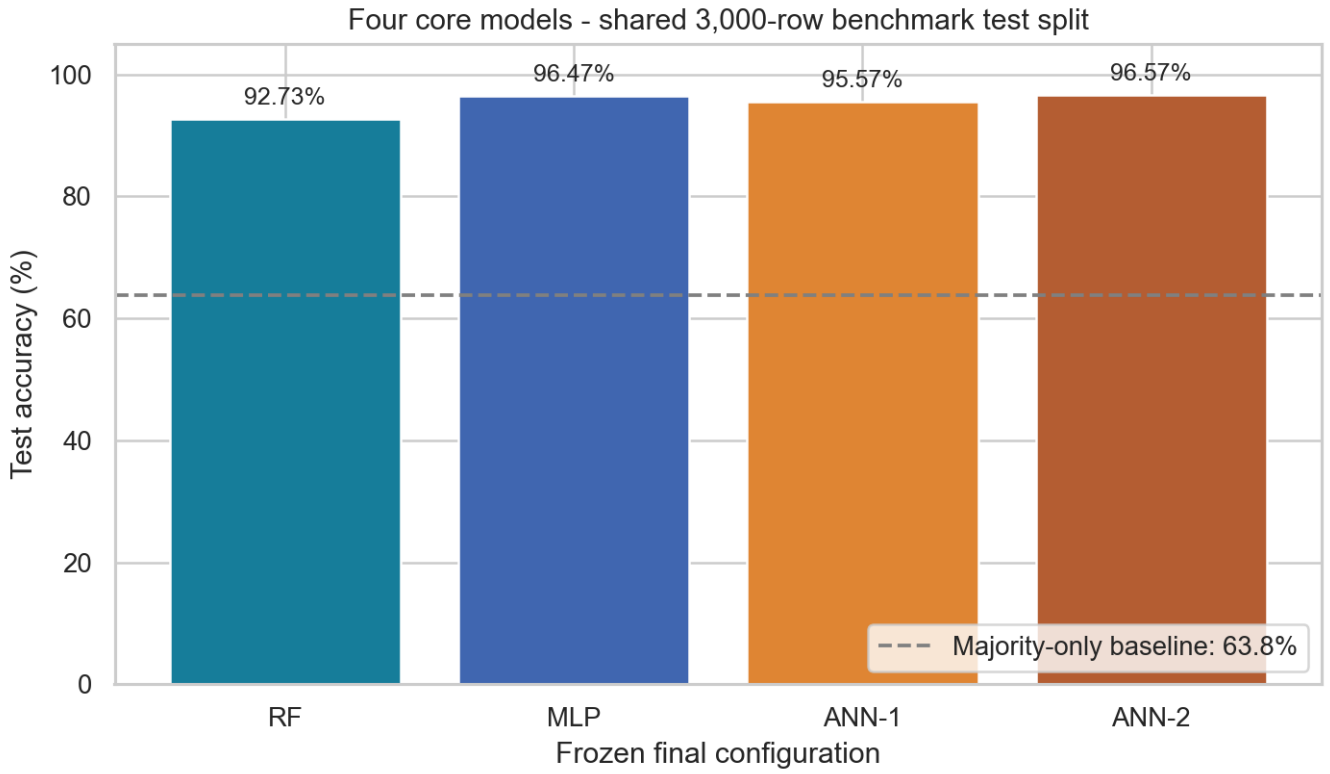


Figure 5. Final test accuracy and the 63.8% majority baseline. ANN-2 exceeds MLP by only three cases, or 0.10 percentage points.

Production recommendation

Do not deploy any model directly to control a real electricity grid from this experiment alone. The data describe a simplified simulation. The split reproduces Portfolio 2 and has been inspected before, so it is a benchmark rather than untouched external validation. New operating regimes, calibrated probabilities, error costs, failure handling and monitored evaluation would be needed. An unstable case called stable may be more consequential than a false alarm, making accuracy alone an inadequate deployment rule.

Classification probability diagnostic

The requested scatter visualisation is supplied as a classification probability diagnostic. Each point is one fixed test case; the horizontal position is the true class with cosmetic jitter, and the vertical position is the predicted probability of unstable. The same 3,000 cases appear in every panel. Jitter does not alter labels or predictions. Red points show errors and the dashed line marks probability 0.5; argmax tie behaviour remains class 0.

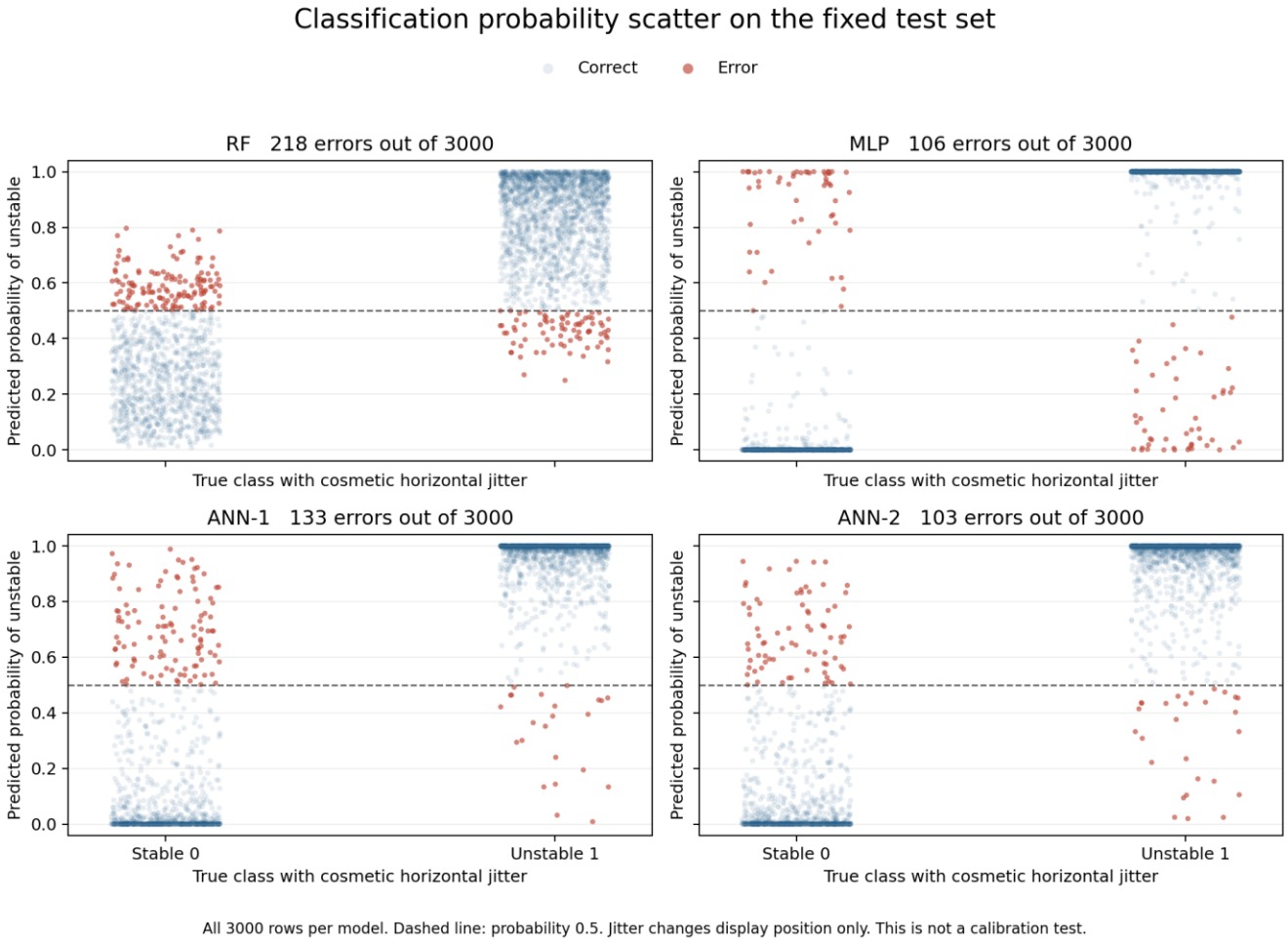


Figure 6. Classification probability scatter for all 3,000 test cases per model. Horizontal jitter affects display only; this is not a calibration test.

Stable cases should generally have low unstable probability and unstable cases high probability. Errors near the boundary show uncertain decisions; errors far from it indicate confident mistakes. This plot does not measure calibration. A continuous actual-versus-predicted diagonal scatter, R squared, RMSE and MAE belong to the regression branch and would be misleading for these categorical labels.

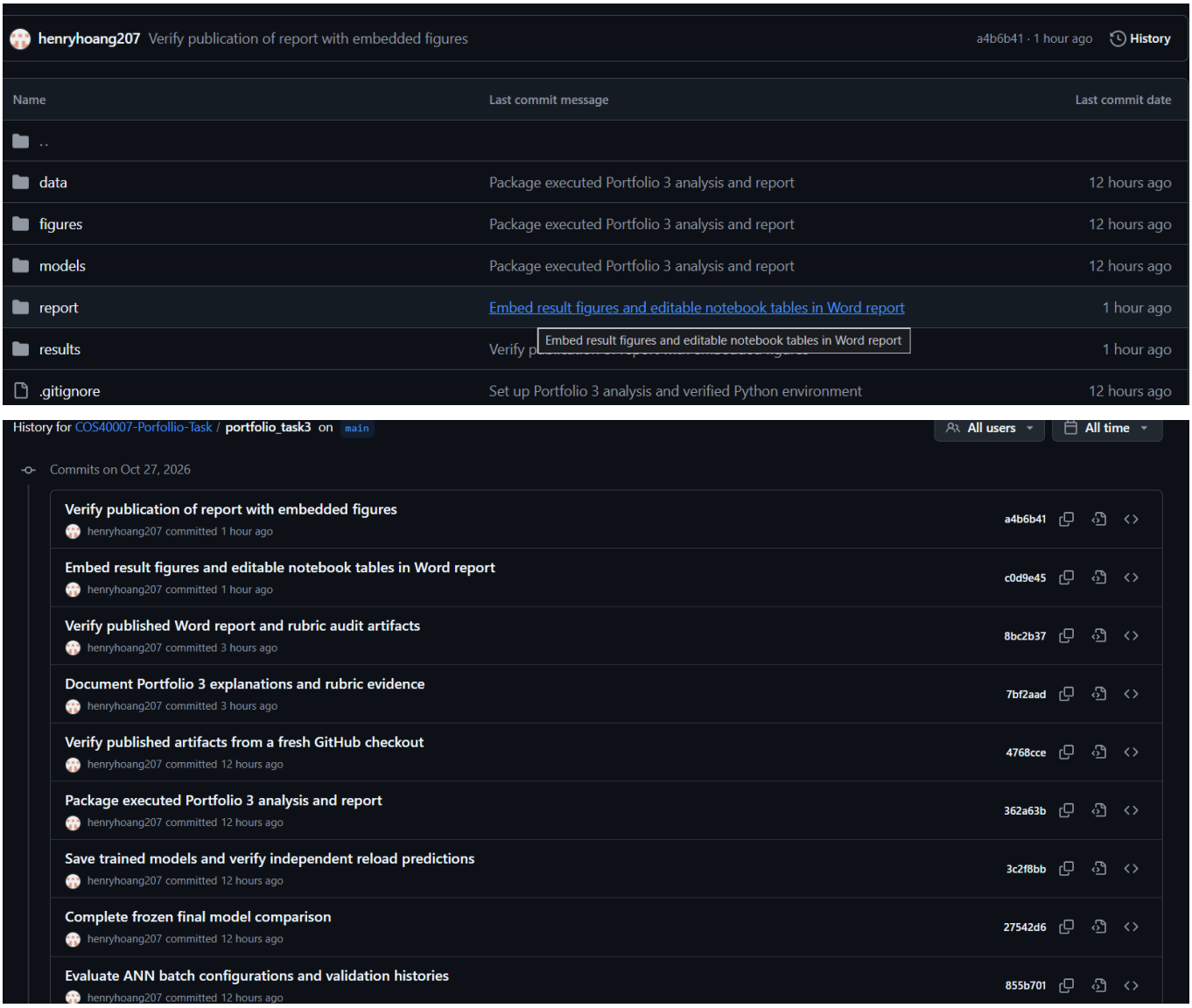
Part D GitHub integration

Repository containing notebook models results and reports

Incremental commit history on main

The original implementation was published in eight genuine stages. The history contains setup, data preparation, traditional models, ANN development, final evaluation, model persistence, report packaging and remote verification. This satisfies the evidence requirement for progressive work; it was not reconstructed as a single bulk implementation commit.

Commit	Meaningful stage
8ba1262	Set up Portfolio 3 analysis and verified Python environment
56d0d69	Prepare grid classification data and verified splits
f190a5f	Compare Random Forest and MLP with five-fold cross-validation
855b701	Evaluate ANN batch configurations and validation histories
27542d6	Complete frozen final model comparison
3c2f8bb	Save trained models and verify independent reload predictions
362a63b	Package executed Portfolio 3 analysis and report
4768cce	Verify published artifacts from a fresh GitHub checkout



S15. Repository and phased history

Part E: GenAI Reflection

1. GenAI Tools Used

OpenAI Codex (model/version not recorded): Brainstorming evaluation ideas, explaining machine learning concepts, reviewing possible errors, checking reproducibility, and drafting this reflection.

The project README also records that Codex generated and executed the working implementation. My disclosure therefore includes that broader implementation assistance alongside the brainstorming and debugging activities described below.

2. Specific Tasks & Prompts

The following prompts are reconstructed examples of the relevant brainstorming and debugging requests. They describe the intended scope of the assistance and are not verified verbatim quotations from a saved chat history.

Task 1: Brainstorming a fair model comparison

Prompt used: "What should I check when comparing Random Forest, MLP and ANN models for electrical grid stability classification? Focus on evaluation fairness, data leakage and suitable metrics."

The main ideas were to use consistent data partitions, distinguish development validation from final testing, and inspect F1 scores and class errors alongside accuracy. These checks are reflected in the common final split and four-model comparison.

Task 2: Reviewing preprocessing for possible errors

Prompt used: "Review my data splitting and scaling steps for potential leakage. Explain any problems and suggest corrections to the affected lines."

This review focuses on fitting each scaler only to the appropriate training rows and preserving the 11-feature order. In cross-validation, scaling belongs inside each fold; validation and test data must not determine the fitted scaling parameters.

Task 3: Checking possible ANN training errors

Prompt used: "Help me diagnose a possible ANN training error. Check the input dimensions, target encoding, output layer and loss function. Explain what to inspect before suggesting a correction."

The relevant checks are 11 input features, integer class labels, two Softmax outputs and sparse categorical crossentropy. These are diagnostic checks; this reflection does not claim that a specific undocumented traceback occurred.

Task 4: Troubleshooting saved-model predictions

Prompt used: "If my reloaded model produces different predictions, what should I check about preprocessing, feature order and model loading? Help me isolate the cause."

The checks cover loading the matching scaler, applying scaling once and preserving feature order. The report records that reloaded class predictions matched exactly and probabilities agreed within the declared numerical tolerance.

3. Code Attribution only for debugging and helping find errors

The entries below identify error checking and diagnostic assistance in Portfolio_Assessment_3_HoangHuuHoan.ipynb. The broader implementation assistance is disclosed in Section 1.

Section 2 - Data audit, continuity and leakage prevention

AI-assisted: Reviewed split and scaling logic for possible leakage and checked feature dimensions and order.

Section 4 - ANN architecture and development experiments

AI-assisted: Checked compatibility between input dimensions, integer labels, Softmax outputs and the loss function.

Section 7 - Persistence and independent reload verification

AI-assisted: Helped check preprocessing consistency, reloaded predictions and numerical tolerances.

4. Learning Reflection

How did GenAI help me learn?

The main learning benefit was connecting modelling choices to the reasons behind them. The preprocessing discussion makes clear why a scaler must learn from training data only, while the ANN checks link integer labels to sparse categorical crossentropy and two Softmax outputs. The comparison also shows why the model with the highest accuracy may not make the fewest important class errors. These explanations give me a clearer basis for interpreting the notebook outputs.

What challenges did it create?

AI suggestions still require checking against the actual code, installed packages and assignment requirements. A plausible explanation can overlook which data fitted a scaler or confuse a development model with the final refit. Generated code can also make it harder to judge how much I understand without assistance. The broader implementation help therefore needs to be acknowledged; successful execution alone does not demonstrate independent understanding.

What specific skills did I develop?

The specific skills reinforced by this work are checking array shapes and label encodings, recognising leakage risks, interpreting training and validation curves, and comparing accuracy with class-level errors. Model persistence adds another useful check: confirming that a saved model, its scaler and its feature order reproduce the expected predictions. My next step is to practise these checks independently and explain the reasoning behind each one.